Dejar que una IA construya una herramienta de ingeniería: qué esperar

Pablo Magariños – *18 de septiembre de 2026*

Palabras clave: Ingeniería con IA, Diseño de aeronaves

Resumen. Construí una primera versión de IDLEDrones, un optimizador de drones de ala fija de competición, con Codex y dos prompts principales: uno para documentar el proyecto y otro para implementarlo. El proceso duró casi tres horas, pero mi trabajo activo fue de menos de 30 minutos. El resultado es una herramienta de ingeniería conceptual que funciona y propone diseños con sentido, aunque todavía hay que tomarla con escepticismo: la física necesita más validación y la interfaz hay que replantearla. De momento no es pública, porque seguimos trabajando en ella.

Qué estoy haciendo

Antes de nada, esto no es **vibe coding** en el sentido de pedir algo en dos líneas y aceptar lo que salga. La IA escribió prácticamente todo el código, pero el peso de mi trabajo estuvo en definir bien el problema, responder a sus preguntas y revisar el resultado con criterio de ingeniero.

Quería construir una herramienta de optimización de aeronaves para las competiciones universitarias de drones de ala fija, en las que cada equipo diseña y construye su propio avión para completar una misión de vuelo: llevar la mayor carga posible, recorrer un circuito lo más rápido posible, o ambas cosas, dentro de un reglamento de diseño. La llamé *IDLEDrones*, de *I Don't Like Engineering: Drones*. Estas competiciones tienen sistemas de puntuación objetivos, al menos en la parte de vuelo; los **informes técnicos** y otras secciones son más subjetivos. Así que todo lo que es objetivo se puede optimizar, si lo simplificas lo suficiente :)

Este artículo no es un tutorial para construir una aplicación desde cero, ni una receta universal: no he hecho ningún máster en programación con IA. Quiero contar qué puedes esperar cuando le das a una IA la capacidad de construir una aplicación casi sin control humano, a través de la historia de cómo salió esta primera versión a partir de dos prompts principales. Evidentemente no puedo incluir aquí todo mi razonamiento, pero creo que he recogido lo esencial, y también qué resultados puede darte un enfoque sencillo, sin una colección interminable de herramientas extra.

Herramientas

Usé exclusivamente Codex, en concreto Codex CLI, es decir, Codex desde la terminal, aunque también hay una aplicación de escritorio. Lo hago así porque cuando empecé no había aplicación para Ubuntu y me acostumbré. Además, me permite abrir varias terminales de Codex en paralelo y ver cómo avanzan todas a la vez: a veces dos en el mismo proyecto, o

unas cuantas más si estoy trabajando en dos proyectos al mismo tiempo.

Trabajo en Ubuntu porque Linux me ahorra algunos problemas de compatibilidad con las bibliotecas científicas de Python. Esta versión en concreto se construyó y verificó en Linux; probablemente se pueda adaptar a Windows o macOS, pero no lo he comprobado. Para gestionar Python en local recomiendo uv: cualquiera puede descargar el proyecto y ejecutar `uv sync` para tener la versión correcta de Python y las dependencias fijadas. Y por mucha IA que uses para programar, el control de versiones sigue siendo imprescindible. Siendo sincero, en este proyecto todo acabó en un único **commit** inicial; es a partir de ahora, iterando sobre esta base, cuando Git se vuelve realmente útil.

La parte que quizá sorprenda a la gente con más experiencia: para este proyecto no creé ningún `AGENTS.md` especial, ninguna **skill** ni ningún MCP. Usé Codex prácticamente **vanilla**, tal como viene. Mi razón es sencilla: OpenAI hace un trabajo mucho mejor que yo optimizando estas herramientas para este tipo de tareas, y demasiadas instrucciones también pueden empeorar su funcionamiento. No le digas de entrada cómo hacer absolutamente todo. Dale cierta libertad y restringe lo que veas que falla.

En otros proyectos sí suelo crear un `AGENTS.md` o un `CLAUDE.md` con instrucciones sencillas cuando las necesito: usar `uv` para gestionar Python, mantener la documentación en `docs/`, escribir el código en inglés aunque yo escriba en español, o cuestionar decisiones que se puedan mejorar. Intento no complicarlo y solo añado reglas que he visto funcionar. No lo hagas antes de saber que lo necesitas. Ese es mi consejo personal.

Prompt 1: le pedí a GPT-6 Astra toda la documentación del proyecto

En el momento de escribir esto (septiembre de 2026), considero que GPT-6 Astra es el estado del arte, junto con Fable 5.1. Usé el esfuerzo *medium*: es un modelo muy potente, y ese nivel suele bastar para documentación sin quemar de una vez todo el límite semanal. Quería que el modelo detallara el proyecto y, sobre todo, que planteara todas las preguntas que necesitara. Un modelo más capaz suele ser mejor detectando fallos, razonamientos débiles e implementaciones alternativas. Después, para la implementación, puedes bajar a un modelo más barato.

Cuando construyo este tipo de herramientas, suelo pedir cosas muy parecidas:

1. Explico el objetivo general. Así, si más adelante me voy por las ramas, el modelo no pierde de vista lo que intento conseguir.
2. Pido un backend en Python y un frontend web. Me ahorra interfaces engorrosas, y FastAPI sirve de puerta de entrada al software de verdad. Python tiene una cantidad absurda de herramientas útiles, como NumPy, JAX o PyTorch, y además es un lenguaje muy presente en los datos de entrenamiento de estos modelos.
3. Explico cómo creo que se podría resolver el problema, a grandes rasgos. Cuanto más concreto seas, y cuanta más libertad le des para rebatirte, más probable es que los dos entendáis el mismo objetivo. Transmitir por escrito una visión concreta es difícil: intenta hacerlo bien, pero deja que el modelo proponga alternativas.

Primero escribí mis ideas en un archivo, `idea_inicial.md`. Estos fueron los dos prompts:

Prompt: idea_inicial.md

> Quiero construir un optimizador de drones de ala fija para competiciones de drones. El objetivo de la competición es recorrer un circuito cumpliendo varios objetivos. El objetivo de la optimización es, por tanto, obtener el dron óptimo para ello. Para esta versión, el circuito de optimización será una recta de 100 metros, un giro cerrado de 20 metros de radio, y lo mismo otra vez, 10 veces. Así que la optimización tiene que dar el dron óptimo para eso.
>
> De momento, había pensado hacer la optimización con un algoritmo genético, iterando sobre diseños de dron. Para las restricciones tienes que darme, en la web, una opción de **drag and drop**. Algunos ejemplos: envergadura máxima, longitud máxima del dron, potencia máxima del motor (para esta prueba suponemos un solo motor), etc. Añade más si se te ocurre alguna restricción clave.
>
> Después, en diseño, tienes que darme opciones: material de las costillas (madera de balsa, aluminio, carbono, etc.), material del revestimiento (fibra de carbono, vinilo, etc.), potencia instalada (recuerda que si instalo X potencia, luego no puedo usar la potencia máxima como restricción; quizá aquí también pueda fijar el peso del motor) y más cosas si se te ocurren. Es el espacio de posibilidades que hay que elegir y que no son triviales, sobre todo materiales y motores.
>
> Tienes que modelar varias cosas: el peso de la batería en función de su capacidad y su tasa de descarga, la aerodinámica del ala con métodos numéricos rápidos pero fiables, la capacidad estructural con métodos rápidos basados en la carga alar, haciendo que el larguero o los largueros del ala la soporten, la masa de la estructura interna simplificada (costillas y largueros), la aerodinámica de alerones y flaps en función de sus dimensiones, y la estabilidad del dron en función de sus dimensiones, distancias, CG, etc. Recuerda también que el objetivo puede ser maximizar la carga de pago, así que tenlo en cuenta durante la optimización.
>
> La idea es optimizar todo lo posible y obtener el dron óptimo para la misión propuesta. El circuito es el que es, y la optimización puede ir a por carga de pago, velocidad, eficiencia energética o tiempo de circuito. Así que déjame ponderar esas cosas.
>
> Quiero que el backend corra en Python de la forma más eficiente posible para acelerar el cálculo, y controlarlo todo desde una web lanzada, probablemente, con FastAPI.

Después, en esa carpeta, donde solo existía ese archivo, le dije esto al modelo:

Prompt: petición de documentación

> Necesito construir un optimizador/simulador profesional para mi equipo de drones de competición. La v1 está en `idea_inicial`. Léelo, analízalo en detalle y escribe en `docs/` todo lo necesario para construirlo, ya que delegarás la construcción en otra IA. Tienes que detallarlo para que funcione bien, haga lo que pido y recoja cosas que quizá no haya descrito: información que falta, enfoques que fallan

> o aspectos que podrían interesarme. Debes preguntarme por todo eso y yo te responderé. Puedes preguntarme mientras planificas o antes.

Como ves, simplemente pienso en voz alta. Suelo escribir lo más rápido posible, porque el cerebro piensa más rápido de lo que escribimos. No me preocupa mucho redactar bien el prompt: intento no gastar espacio mental en una redacción perfecta.

La IA creó `docs/` y se puso manos a la obra. En tres o cuatro ocasiones volvió con tandas de preguntas: órdenes de magnitud de las dimensiones, si quería modelar el despegue, cómo debía ser la fórmula a maximizar, etc. La clave aquí es estar atento y pensar a largo plazo: qué problemas aparecerán cuando el software esté terminado, si de verdad será útil, o si el alcance es tan ambicioso que se volverá inmanejable. Las respuestas a estas preguntas influyen mucho en que el resultado sea bueno o malo.

Gracias a esas preguntas se me ocurrieron cosas que no había incluido en el prompt inicial. La que más me gustó fue añadir un análisis de Monte Carlo de las soluciones. Como te puedes imaginar, la suerte pesa mucho en una competición. La v1 varía el viento y su dirección; en el futuro me gustaría añadir también el rendimiento de los equipos rivales. Muchas competiciones tienen puntuaciones que dependen del mejor resultado en una categoría, por ejemplo la velocidad máxima. Eso puede desplazar el óptimo real y abre la puerta a la teoría de juegos: el dron óptimo puede depender de si todos los demás maximizan una puntuación o varias. Eso da para otro artículo, pero la teoría de juegos y el modelado de los rivales son temas muy interesantes.

Hay algo en lo que no insistí lo suficiente en el prompt inicial y que debería haber pedido desde el principio: tests explícitos para validar la física. El plan y la implementación acabaron incluyéndolos, pero en una herramienta de simulación compensa exigir desde el primer momento tests unitarios para cada modelo, comparaciones con datos reales y tests de sistema que comprueben la coherencia entre las subsimulaciones. También puede ayudar comparar formulaciones o solvers independientes y estudiar los márgenes de error esperables según la literatura, aunque que dos solvers coincidan no sustituye a la validación experimental.

Prompt 2: poner a GPT-5.6 Sol a implementarlo

Una vez terminada la documentación, le pedí a GPT-6 Astra que escribiera un prompt para una sesión nueva y cerré la original. La idea era que el nuevo agente supiera ejecutar el plan sin perderse en toda la documentación. No incluyo ese prompt porque lo escribió la propia IA, y así ahorro espacio.

Ejecuté GPT-5.6 Sol con esfuerzo *xhigh* para que fuera más minucioso con los tests y con la búsqueda de información. Quizá *high* habría bastado, pero quería algo más potente. Soy partidario de invertir mucho esfuerzo cuando todavía no existe nada, porque después es más difícil arreglar lo que ya está construido.

Como referencia de un punto intermedio: a los 40 minutos de ejecución estaba inspeccionando archivos PNG generados al renderizar los informes PDF. Esa es una de sus ventajas: al ser multimodal, puede comprobar visualmente lo que produce. Por eso es tan interesante que estos modelos puedan leer imágenes.

A la hora y media (sí, es bastante, en parte por el *xhigh*), lo había visto borrar código y reescribirlo varias veces. Eso demostraba que estaba iterando, probando y construyendo el proyecto poco a poco, en lugar de escribir una única versión y darla por terminada.

Debo aclarar algo sobre los límites de uso. En el momento de escribir esto (septiembre de 2026), uso Codex con el plan Plus y, en mi caso, no tengo el límite de cinco horas, solo el semanal. Eso me permite lanzar tareas extremadamente largas; con Claude, en cambio, no puedo hacerlo igual. Sé que otras personas en Codex sí tienen el límite de cinco horas, y no pasa nada: un proyecto así sigue siendo posible con límites más cortos, solo que lleva más tiempo. Puedes retomar la tarea cuando se renueve tu uso, o mantener un archivo de tareas e irlo actualizando. Además, cuanto más larga es una conversación, más contexto arrastra y mayor puede ser el consumo. Mantener tareas pequeñas y bien acotadas puede ahorrar uso al trabajar con contextos más cortos.

Todo el proceso duró casi tres horas y consumió aproximadamente el 40 % de mi uso semanal de Codex. Mi tiempo de trabajo activo fue de menos de 30 minutos: sobre todo escribir los prompts, responder preguntas, revisar el progreso y comprobar el resultado. El resto fue el modelo trabajando de forma autónoma.

Resultado (en resumen)

El resultado es una aplicación local para Linux, con backend en Python y control web, capaz de evaluar y optimizar aviones convencionales, alas volantes y canards. No se limita a variar una geometría: cierra de forma conjunta la aerodinámica, la estructura, la masa y el CG, la estabilidad, las superficies de control, el alojamiento de la carga de pago, el motor, la hélice, la batería y la misión. Si un diseño no puede despegar, trimar, soportar las cargas, entregar la potencia o completar el circuito con margen, se descarta y se guarda el diagnóstico.

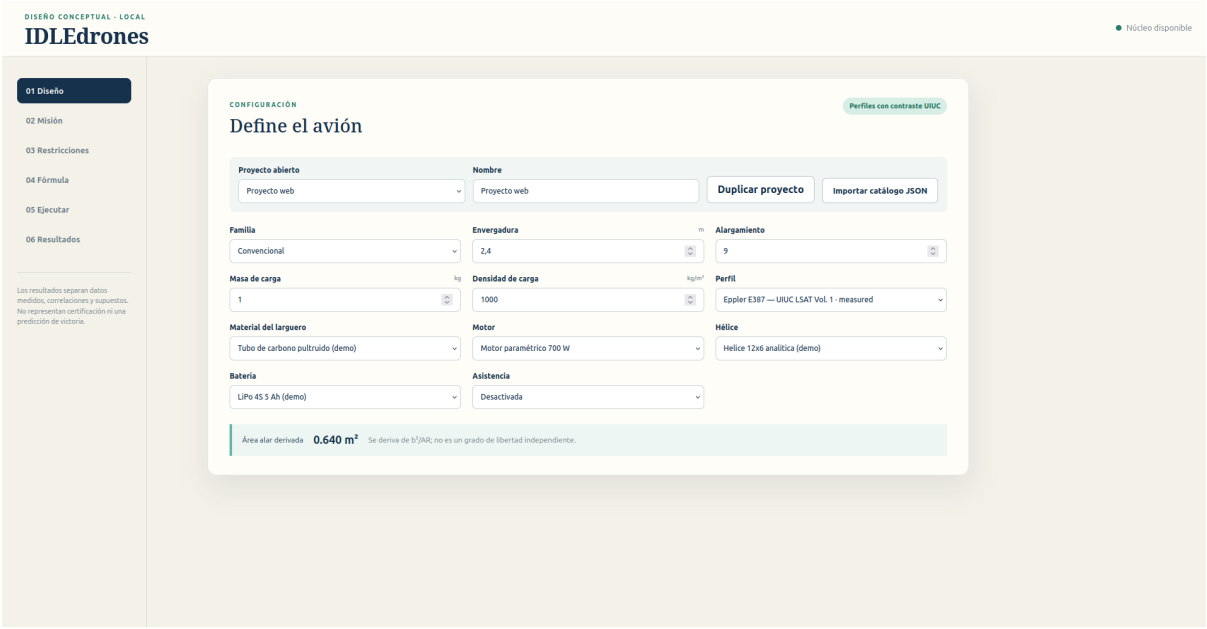


Figura 1: Editor de diseño de IDLEDrones

El editor define la familia, la geometría, la carga de pago y los componentes principales de la aeronave.

Los nombres de componentes como «LiPo 4S 5 Ah (demo)» o «Motor paramétrico 700 W» son etiquetas genéricas que la IA dio a las entradas del catálogo, no productos comerciales.

El optimizador combina variables continuas con componentes de catálogo, compara las familias por separado y refina los mejores candidatos con modelos de mayor fidelidad. Desde la web puedes editar la fórmula de puntuación y las restricciones, lanzar evaluaciones u optimizaciones, simular viento con Monte Carlo, cancelar y reanudar trabajos, comparar resultados y exportarlos en JSON y PDF. Cada ejecución guarda su configuración, semillas, catálogos, versiones de los modelos y hashes para poder reproducirla y auditarla.

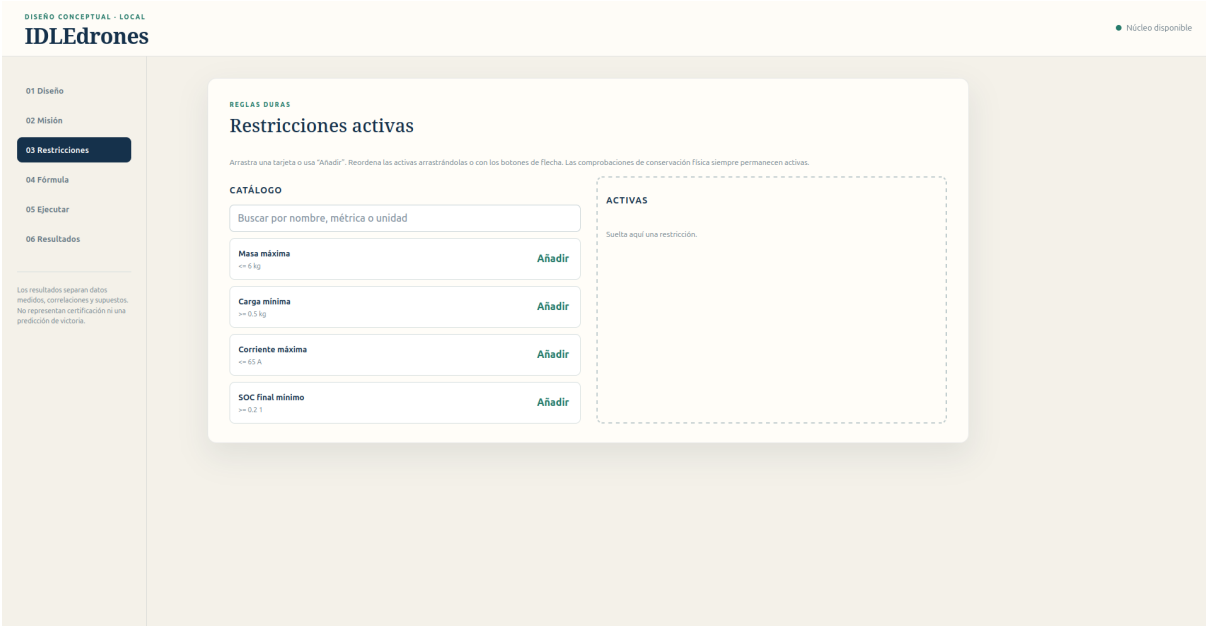


Figura 2: Editor de restricciones de IDLEDrones

Las restricciones se añaden desde un catálogo arrastrando tarjetas; las comprobaciones físicas básicas siempre siguen activas.



Figura 3: Resultado de una evaluación en IDLEDrones

La aplicación muestra las métricas del candidato evaluado, sus márgenes y los componentes seleccionados. Identificadores como `motor-1000w-demo` *o* `lipo-4s-3ah-demo` *son, de nuevo, nombres genéricos que asignó la IA.*

Análisis

Una de las cosas más fascinantes es que incorporó datos reales y públicos de fabricantes y de repositorios experimentales, además de ecuaciones y correlaciones físicas. No es un simple juguete: es una herramienta de ingeniería en una fase temprana. Aun así, las comparaciones actuales son sobre todo por componente, y todavía no validan la precisión de la aeronave en su conjunto.

Los drones que propone tienen sentido desde el punto de vista de la ingeniería, pero de momento conviene ser escéptico, porque la herramienta muestra el ganador y poco más. Sin pruebas de que realmente sea el mejor, «es el óptimo» sigue siendo una afirmación discutible.

Otra limitación de la verificación es que, sin que yo lo supiera durante la ejecución, el entorno no tenía acceso a un navegador gráfico. El flujo web se comprobó con tests de la API y de la aplicación compilada, pero no con una sesión interactiva real en un navegador. La revisión visual se limitó a los PDF renderizados.

En un punto intermedio conté 34 tests. La entrega terminó con 47 tests pasando, que sigue siendo una cobertura modesta para una herramienta que quisieras considerar crítica. Si empezara de nuevo, insistiría antes en construir los modelos con más rigor de ingeniería. Probablemente pasaría de **documentación → herramienta** a:

1. Documentación.

2. Contraste científico de fórmulas y datos.
3. Implementación incremental con tests.
4. Validación integrada.

Probablemente mantendría GPT-6 Astra para la documentación y usaría GPT-5.6 Sol para las otras tres fases.

Otra cosa que mejorar es que todavía quedan algunas constantes heurísticas dentro de los modelos, como el tamaño de la malla de velocidades o la discretización de los giros. No todo está **hardcodeado**: los catálogos, los componentes, los parámetros de la misión, las restricciones y las principales opciones físicas son editables. Aun así, convendría identificar y centralizar las constantes que afectan a la fidelidad numérica, para poder revisarlas y estudiarlas con facilidad.

Como punto de partida el resultado es impresionante, pero todavía queda trabajo para que sea realmente útil. La parte más débil es la interfaz: mezcla la edición de una aeronave concreta con la definición del espacio de una optimización. Por ejemplo, pedir de entrada la envergadura y las dimensiones sugiere que tienes que fijar la aeronave antes de optimizarla, cuando deberían ser rangos, límites o variables que decida el optimizador. También te deja elegir una familia base mientras la optimización puede explorar las tres, sin explicar bien la relación.

La base está, pero queda mucho trabajo

La herramienta funciona, pero deja varias direcciones claras de mejora:

- **Aerodinámica y estructuras.** Usa modelos conceptuales razonables para explorar diseños, pero le faltan perfiles y efectos aerodinámicos más detallados, así como estructuras fabricables con uniones, adhesivos y detalles locales.
- **Validación.** Necesitaría más datos de banco y de vuelo para comprobar la aeronave completa, no solo sus modelos por separado.
- **Código y tests.** La arquitectura ya está dividida en módulos, pero habría que robustecer sus interfaces y ampliar los tests de regresión y de integración.
- **Optimización y cálculo.** Si hubiera que explorar espacios mucho más grandes, se podría estudiar el cálculo distribuido, la vectorización o las GPU; añadir una GPU no basta si los modelos no están adaptados a ella. El optimizador también debería justificar su resultado, con una comparación frente a los candidatos descartados, la sensibilidad a los pesos de la puntuación y si la optimización ha convergido.
- **Interfaz.** Necesita una separación clara entre «analiza esta aeronave» y «encuentra la aeronave óptima».

No son funcionalidades que añadir por añadir, sino posibles caminos para convertir esta primera versión conceptual en una herramienta más fiable.

En resumen, la herramienta me ha ahorrado muchísimo trabajo, pero ahora mismo está muy lejos de lo que pretendía ser. La interfaz y la física todavía tienen que mejorar y, sobre todo, hay que reconducir parte de lo construido hacia algo más serio y fiable.

Conclusiones y consejos

No tengas miedo de pedir

La IA es muy potente, más de lo que queremos creer. No tengas miedo de pedir algo por si la IA no está a la altura. Preocúpate, sobre todo, de documentar bien cómo quieres que funcione.

Tampoco tengas miedo de pedir por el vértigo de que la IA lo haga mejor que tú. Cuanto antes aceptes que puede hacer en segundos cosas que a ti te llevarían días o semanas, mejor.

Sé preciso y riguroso lo antes posible

Por mucho contexto que puedan manejar los modelos, el mejor momento para hacer cambios es al principio. Merece la pena dedicar tiempo a la documentación y la planificación. Es entonces cuando el proyecto es más compacto; después habrá una cantidad enorme de ruido de implementación.

Hay que encontrar un punto medio entre este consejo y el anterior. Cuanto más sepas del tema, más preciso deberías ser. Cuantos menos detalles des, más libertad tiene el modelo, y eso puede salir bien o mal. En mi opinión, compensa ser preciso en lo que sabes y dejarle libertad para opinar en lo que no.

No construyas por construir

Tener una herramienta tan potente crea la tentación de construir cosas cada vez más grandes. Me ha pasado que, antes de darme cuenta, había añadido funcionalidades innecesarias que solo inflaban el resultado y convertían la herramienta en un ladrillo inmanejable. No pierdas de vista lo que intentas hacer. Diseña una estructura que pueda crecer y te permita añadir cosas en el futuro, pero empieza por algo pequeño. Piensa en grande, actúa en pequeño.

Reflexión

Es fundamental seguir estudiando y aprendiendo cosas nuevas, porque por mucho que avance la IA, nunca podrás pedir algo que no sabes que existe. Parece una tontería, pero importa. Al menos por ahora, somos nosotros quienes decidimos qué hacer: podemos delegar el **cómo** en la IA, pero seguimos decidiendo el **qué**. No puedes querer un dron de ala fija si no sabes que existen los drones de ala fija. Si solo dices «dron», la IA puede inclinarse por un cuadricóptero.

Por otro lado, el espacio de posibilidades es prácticamente infinito. Un problema puede tener infinidad de soluciones y formas de abordarlo. A medida que añades restricciones, objetivos concretos y contexto, vas cerrando ese espacio. Pedirle a una IA en cinco líneas la solución óptima a tu problema casi nunca funcionará, porque le falta una cantidad enorme de contexto que tú sí tienes. Ser preciso al plantear el problema y aprender sobre el tema seguirá siendo útil. No es solo una cuestión de la capacidad de la IA, sino del espacio de posibilidades y de la información disponible. Los modelos tendrán cada vez más intuición, pero no pueden reconstruir fielmente un contexto que nunca les diste.